

Modules avancés

Tour d'horizon des modules Node.js de production

Module 7/7

Objectifs du module

Durée : 3h30

- Tracer le contexte asynchrone (Asynchronous Context Tracking, Async Hooks) et gérer Process Events et Errors
- Exploiter Child Process, Clusters et Worker Threads, ainsi que Web Streams API, Buffer et Stream
- Survoler Crypto, TLS (SSL), Web Crypto API, Performance Measurement APIs, Events avec RxJS, File System et les C++ Addons / Node-API

Plan du module

1. Contexte asynchrone, processus et événements
 - AsyncLocalStorage et Async Hooks, Process Events et Errors, Events avec RxJS (survol)
2. Concurrency et flux de données
 - Child Process, Clusters, Worker Threads ; File System, Buffer, Stream, Web Streams API
3. Sécurité, mesure et extension native
 - Crypto, TLS (SSL), Web Crypto API ; perf_hooks ; C++ Addons / Node-API (survol)

1. Contexte asynchrone, processus et événements

Le problème de la corrélation

Concept : suivre une même requête HTTP à travers toutes ses opérations asynchrones (logs, accès base, appels externes) sans propager manuellement un identifiant.

Sans contexte :

- passer `requestId` en paramètre partout
- pollue toutes les signatures
- oubli facile dans le code tiers

Avec `AsyncLocalStorage` :

- un store par chaîne d'exécution asynchrone
- accessible depuis n'importe quelle fonction
- aucun paramètre à propager

AsyncLocalStorage : principe

`AsyncLocalStorage` conserve une valeur le long de la chaîne d'appels asynchrones initiée par `run()`. Chaque requête a son propre store isolé.

```
import { AsyncLocalStorage } from "node:async_hooks";

const als = new AsyncLocalStorage();

// Middleware Express : ouvre un store par requête
function context(req, res, next) {
  const store = new Map();
  store.set("requestId", req.headers["x-request-id"] ?? crypto.randomUUID());
  als.run(store, () => next());
}

// N'importe où dans la pile d'appels
function log(message) {
  const id = als.getStore()?.get("requestId");
  console.log(JSON.stringify({ requestId: id, message }));
}
```

Async Hooks : le mécanisme sous-jacent

Concept : `async_hooks` expose le cycle de vie des ressources asynchrones (`init`, `before`, `after`, `destroy`). `AsyncLocalStorage` est construit dessus.

À retenir :

- API bas niveau, rarement utilisée directement
- coût en performance non nul
- réservée à l'instrumentation/APM

```
import { createHook } from "node:async_hooks";

createHook({
  init(asyncId, type, triggerId) {
    // ressource asynchrone créée
  },
  destroy(asyncId) {
    // ressource détruite
  },
}).enable();
```

Process Events et Errors

Concept : `process` est un EventEmitter. On y branche signaux système, erreurs non capturées et arrêt propre (graceful shutdown).

```
process.on("SIGTERM", () => {
  server.close(() => {
    db.end();
    process.exit(0); // arrêt propre une fois les connexions drainées
  });
});

process.on("uncaughtException", (err) => {
  console.error("Erreur non capturée", err);
  process.exit(1); // l'état est corrompu : on redémarre
});

process.on("unhandledRejection", (reason) => {
  console.error("Promesse rejetée non gérée", reason);
});
```


Events avec RxJS (survol)

Concept : RxJS modélise des flux d'événements comme des `Observable`, avec composition (`map`, `filter`, `debounceTime`) et désinscription explicite.

EventEmitter (Node natif) :

- simple, intégré
- pas de composition d'opérateurs
- gestion manuelle du désabonnement

```
import { fromEvent } from "rxjs";
import { throttleTime, map } from "rxjs/operators";

fromEvent(emitter, "data")
  .pipe(throttleTime(100), map((e) => e.size))
  .subscribe(console.log);
```

À réserver aux flux complexes.

Démonstration

Voir `demos/demo-07.md`

Tracer une requête de bout en bout avec AsyncLocalStorage (requestId dans tous les logs), puis signer et vérifier une donnée avec le module Crypto.

2. Concurrency et flux de données

Child Process : exécuter du code externe

Concept : `child_process` lance des processus séparés. Trois API selon le besoin.

- `spawn` : flux en continu (gros volumes, streaming stdout)
- `exec` : commande courte, sortie bufferisée en mémoire
- `fork` : sous-processus Node avec canal IPC

```
import { spawn } from "node:child_process";

const ffmpeg = spawn("ffmpeg", ["-i", "in.mp4", "out.webm"]);
ffmpeg.stdout.on("data", (d) => process.stdout.write(d));
ffmpeg.on("close", (code) => console.log("fini", code));
```

Clusters vs Worker Threads

Concept : deux façons d'utiliser plusieurs cœurs. Le bon choix dépend de la nature de la charge.

Critère	cluster	worker_threads
Modèle	processus séparés	threads dans le process
Mémoire	isolée par processus	partagée possible (SharedArrayBuffer)
Usage type	scaler des serveurs HTTP I/O	calcul CPU intensif

Règle : I/O et montée en charge HTTP → cluster (ou un orchestrateur). Calcul lourd qui bloque l'event loop → worker_threads.

Worker Threads en pratique

`worker_threads` déporte un calcul bloquant hors de l'event loop principal. Le thread principal reste disponible pour servir les requêtes.

```
// main.js
import { Worker } from "node:worker_threads";

function calcul(n) {
  return new Promise((resolve, reject) => {
    const w = new Worker("./worker.js", { workerData: n });
    w.on("message", resolve);
    w.on("error", reject);
  });
}

// worker.js
import { parentPort, workerData } from "node:worker_threads";
const result = fibonacci(workerData); // calcul CPU lourd
parentPort.postMessage(result);
```

Buffer et Stream

Concept : `Buffer` représente des octets bruts. `Stream` traite les données par morceaux, sans tout charger en mémoire.

Streams Node :

- `Readable`, `Writable`, `Duplex`, `Transform`
- `pipeline()` pour chaîner avec gestion d'erreur
- backpressure géré automatiquement

```
import { pipeline } from "node:stream/promises";
import { createReadStream, createWriteStream } from "node:fs";
import { createGzip } from "node:zlib";

await pipeline(
  createReadStream("data.json"),
  createGzip(),
  createWriteStream("data.json.gz")
);
```

Web Streams API et interop

Concept : la Web Streams API (`ReadableStream` , standard navigateur) est disponible dans Node. On convertit entre streams Node et Web Streams.

```
import { Readable } from "node:stream";

// Stream Node -> Web Stream (ex. pour une Response Fetch/WHATWG)
const webStream = Readable.toWeb(nodeReadable);

// Web Stream -> Stream Node
const nodeStream = Readable.fromWeb(webReadableStream);

// Express 4 attend des streams Node : on pipe directement
nodeReadable.pipe(res);
```

Utile pour l'interopérabilité avec `fetch` , les Response WHATWG et le code isomorphe.

File System

Concept : `node:fs` offre des API synchrones, callback et promesses. Privilégier les promesses et le streaming pour les gros fichiers.

```
import { readFile, writeFile } from "node:fs/promises";

const data = await readFile("config.json", "utf8");
await writeFile("out.json", JSON.stringify(parsed), "utf8");

// Gros fichiers : streamer plutôt que readFile (évite la saturation mémoire)
import { createReadStream } from "node:fs";
createReadStream("big.csv").pipe(res);
```

3. Sécurité, mesure et extension native

Crypto : hash, hmac, signatures

Concept : `node:crypto` couvre hachage, HMAC (intégrité avec secret partagé) et signatures asymétriques (authenticité avec paire de clés).

```
import { createHash, createHmac } from "node:crypto";

// Empreinte d'intégrité
const hash = createHash("sha256")
  .update(payload)
  .digest("hex");

// HMAC : intégrité + secret
const mac = createHmac("sha256", secret)
  .update(payload)
  .digest("hex");
```

```
import { sign, verify, generateKeyPairSync }
  from "node:crypto";

const { privateKey, publicKey } =
  generateKeyPairSync("ed25519");

const signature = sign(null, data, privateKey);
const ok = verify(null, data, publicKey, signature);
```

TLS (SSL) et Web Crypto API (survol)

TLS (SSL) :

- chiffrement du transport (HTTPS)
- géré le plus souvent par un reverse proxy en prod
- `node:tls` pour les cas spécifiques (mTLS)

Web Crypto API :

- `crypto.subtle`, standard, asynchrone (Promise)
- même API navigateur et Node
- à préférer pour le code isomorphe

```
const digest = await crypto.subtle.digest(
  "SHA-256",
  new TextEncoder().encode("data")
);
```

Performance Measurement APIs

Concept : `node:perf_hooks` mesure des durées avec précision et observe des événements de performance (`PerformanceObserver`).

```
import { performance, PerformanceObserver } from "node:perf_hooks";

const obs = new PerformanceObserver((list) => {
  for (const e of list.getEntries()) {
    console.log(`${e.name} : ${e.duration.toFixed(2)} ms`);
  }
});

obs.observe({ entryTypes: ["measure"] });

performance.mark("start");
await traiterRequete();
performance.mark("end");
performance.measure("requete", "start", "end");
```

C++ Addons / Node-API / Embedder API (survol)

Concept : étendre Node avec du code natif (C/C++) ou embarquer le moteur dans une autre application.

Quand et pourquoi :

- besoins de performance extrême (cryptographie, calcul)
- réutiliser une bibliothèque C/C++ existante
- accès matériel/système non exposé en JS

Comment :

- Node-API (`napi`) : ABI stable entre versions
- `node-gyp` pour la compilation
- coût : portabilité, maintenance, build

Réserver aux cas justifiés et mesurés.

Travaux pratiques

Fichier : `tp/tp-07.md`

Composer un service combinant Worker Threads (calcul), Crypto (signature du résultat) et Web Streams API (exposition du flux), le tout dans une route Express.

Difficulté 4/5 — durée estimée 1h30.

Récapitulatif

- AsyncLocalStorage corréle les logs d'une requête sans propager d'identifiant ; Async Hooks en est le socle bas niveau
- `process` pilote signaux, erreurs et arrêt propre
- `cluster` scale l'I/O, `worker_threads` déporte le calcul CPU ; `Buffer/Stream` et `Web Streams` traitent les données par morceaux
- `Crypto` signe et vérifie ; `perf_hooks` mesure ; les add-ons natifs restent un dernier recours

Cheatsheet : `cheatsheet/module-07.md`

Questions ?

Prochaine étape : Synthèse et clôture de la formation
